

ASSEMBLYAI VOICE AGENT HACKATHON · SEPTEMBER 2026

Voice Incident Reporter

Voice in, evidence out. Post-visit incident reports with a spoken read-back loop and published accuracy — the technician just talks; the agent interviews, fills the report live via tools, and reads critical values back out loud until the record is provably correct.

Team el-informatico · 10 real measured sessions against the live API · failures published · audio never stored
github.com/el-informatico/field-service-voice-logger · field-service-voice-logger.vercel.app

THE PROBLEM

The 45-minute problem

Field technicians spend most of an hour per job on paperwork — and dictation alone cannot fix it.

30–60 min

per job writing up the visit — often after hours, in a truck, from memory. Notes get lost, reports get thin, nothing is auditable.

- **Voice-to-text exists** — but dictation cannot tell a report from a guess.
- **production** vs **staging** servers · rack **A3** vs **A8** · 3/4" vs 3/8" valves — all sound close enough to end up on an invoice.
- Conversational copilots exist in adjacent verticals — **nobody applies the spoken confirmation loop to field-service records.**

The wedge: a voice-filled record you can invoice against — which means the captured values must be **confirmed out loud**, not just transcribed.

Nobody in this gallery publishes accuracy

We audited the hackathon gallery before building the claim, and re-checked it as it grew.

61 → 63 → 94 submissions audited / counted

Full gallery audits on **2026-09-15** (61) and **2026-09-17** (63) with cited quotes; the dashboard showed **94 live** at our 09-21 check, spot-checked. Evidence with URLs and access dates in the repo.

Zero publish measured extraction accuracy

The closest entry publishes a scenario pass-rate and latency percentiles — not per-field precision/recall vs ground truth, not WER, not confirmation-loop precision. A 09-21 spot-check of the new top-voted entries found none either.

Every demo fills a form or dispatches

Voice demos we could find either fill forms or execute operations autonomously. **None rescue capture errors by speaking them back** before they count.

So the differentiator is not "voice" and not "forms" — it is **the confirmed record**: read-back of confusable values, an audit trail per field, and **published, reproducible numbers, failures included**.

WHAT IT DOES

After the visit, the technician just talks

One voice session replaces the write-up — and ends as evidence, not as audio.

1

Consent first

Visible consent screen before the mic ever opens; session only on an explicit tap.

2

Live interview

Raw WebSocket to the AssemblyAI Voice Agent API. One thing at a time; VAD tolerant of working pauses; barge-in enabled.

3

Card fills live

Every statement becomes a client-side JSON-Schema tool call. Each field carries its own audit trail: which tool set it, when, voice-confirmed or hand-edited.

4

Read-back loop

Critical values — services, severity, timeline hours — are spoken back and confirmed before they count.

5

Evidence out

CSV for the ticket system, print-ready PDF, and a JSON timeline artifact powering replay + metrics. Audio discarded by design.

Why after the visit: live voice during the repair was tried and **gated out by evidence** — a pre-registered 10 dB noise gate failed every criterion (runbook and results committed). The pivot is documented, not hidden.

What you can watch

Three artifacts, each labeled for what it is — nothing synthesized is presented as real session audio.

Pitch video — 3:40, EN, captions burned

Problem → live UI filling via tools → read-back rescue → metrics table. On-screen label and narration both state it is a **deterministic replay of the scripted session on the mock channel**; the numbers shown come from the real API runs. Narration is synthetic TTS, disclosed as such.

Real-session clip — 1:06

A real live-API session: the incident card filling from actual tool calls. **Silent by design** (audio is never stored); disclosure bands burned in state the operator audio was a scripted wav. Recorded on the original Spanish-language build.

Live demo

The deployed app runs the **deterministic mock channel** — no API key on the public server, by design. Same engine, same tools, same artifacts as the real channel; real sessions run locally with a one-use token.

Judge-verifiable from the repo alone: one command (**`npm run smoke:mock`**) runs sessions → artifacts → metrics deterministically, no API key, no cost.

One engine, two channels, zero npm dependencies

Static browser app with no build step; a DOM-free session engine shared by browser and Node — which is what makes the pipeline CI-able.

Browser — static, no build

- **Raw WebSocket** to the AssemblyAI Voice Agent API — contract verified from official docs; **no SDK exists for the web**, so we wrote the client
- **Tools defined client-side** with JSON Schema + catalog enums — the agent can only pick services that exist
- Incident state + per-field audit trail + artifact builder
- **Audio: memory only** — never disk, never upload

Serverless — Vercel

- `api/token` — issues **one-use temp tokens**; the API key never reaches the browser
- `api/sessions` — stores transcript + tool state, **never audio**; rejects artifacts claiming otherwise

Mock channel — CI

- Same interface as the real channel; deterministic rule-based interviewer
- **`npm run selftest` + `npm run smoke:mock`** = the whole CI
- Node 22, ESM, zero runtime deps

The incident product was a pivot: **~70% of the code** (channel, engine, artifact, metrics, export) is shared with the original work-order variant, which remains in the repo with its noise-gate evidence.

Confusables are caught twice

The schema rejects what cannot exist; the spoken loop rescues what exists but is wrong.

Catch 1 — catalog enum in the schema

A service the catalog does not define is **rejected before the read-back even asks**. The agent cannot invent entries; services set precision across the 10 real sessions: **100% (12 TP / 0 FP)**.

Catch 2 — the spoken read-back

Real-but-wrong values survive the schema — that is what the loop is for. Read-back precision (read-backs that caught a real error): **62.1% over 29 read-backs** in real dialogue.

*"did you say **production**? the catalog also has **staging** — which one was it?"*

A seeded capture error, rescued in a real session

Rack **A8** captured → disambiguation read-back naming both catalog services → operator confirms **A3**. Recorded in the session artifact (committed as evidence): gate_R5d, script i4.

MEASURED RESULTS

N=10 real sessions — the whole table

Real voice sessions against the live AssemblyAI Voice Agent API: scripted operator at natural pace, seeded ground truth, quiet post-visit dictation. 5 sessions on prompt v3 + 5 on v4.

METRIC	VALUE	N	CONDITION
Turn completion (scripted turns transcribed)	7–10 of 9–11 per session	10 sessions	quiet dictation
WER, matched pairs	0.231 (0.164–0.382/session)	2,134 ref words	quiet dictation
End-of-speech → tool call, p50 / p95	1,554 ms / 4,810 ms	53 tool turns	real API
Severidad exacta	6/10 sessions	10	read-back loop
Servicios afectados set precision	100% (12 TP / 0 FP)	10	catalog enum
Servicios afectados set recall	70.6% (5 FN)	10	see notes
Timeline horas exactas	19/35 GT events (54.3%)	10	hora exacta; see notes
Confirmation-loop precision (read-backs → correction)	62.1%	29 read-backs	real dialogue
Barge-in respected (interrupt → reply cut ≤500 ms)	0/3 (1,338 / 9,034 / 20,501 ms)	3 designed interrupts	hour-correction script

Definitions exact and reproducible from metrics/ in the repo · four session artifacts committed as evidence (docs/evidence/gate/) · honest notes on slide 10.

A measured prompt fix, not a vibe

Same scripts, same conditions — only the interview prompt changed, and the numbers moved.

The collapse (v3)

Pure-narrative capture — continuous dictation, no pauses, no markers — ended in **1 tool call and 0/8 timeline hours, twice in a row**. Services recall 37.5%, hour-exact timeline 35.3%, severity 2/5, EOS→tool p50 1,775 ms.

The fix (v4) — a "registrar" reframe

One rule set: no spoken datum goes unregistered, **one tool call per datum**, read-back grouped at the close, hour-conversion few-shots ("eighty" → 8:50).

The result

Services **100%/100%** precision/recall (9 TP / 0 FP / 0 FN) · timeline hours **72.2%** (13/18) · severity **4/5** · EOS→tool p50 **1,139 ms**. The collapsed script now registers 10–15 tool calls and 7/8 hours.

WER is 0.231 in both halves — the fix changed tool-call discipline, not hearing. That is the point: the loop, not the transcriber, carries correctness.

And the discipline about the discipline: a v5 attempt (three targeted edits) regressed in its only measured run — read-backs spoken without tool calls — and was **fully reverted**. The two-iteration budget was pre-registered and exhausted; the variance is documented, not iterated away.

The limits, published with the numbers

Everything below is in the README and the submission text — the failures are part of the deliverable.

- **WER 0.231 on matched pairs** — roughly one word in four. Not hidden; it is the reason the product is a confirmation loop and not a transcriber.
- **The rig is rules-only:** scripted operator utterances at natural pace, seeded ground truth, no free-form human turns. Reproducible numbers — and no claim of spontaneous-conversation performance.
- **2 of 10 sessions hit the 300 s driver watchdog** and lost their final turns — one severity miss is the correction turn that never played.
- **0 of 10 reached enviar_reporte** — post-audit, that was a driver close-out race, not an agent refusal: fixed in the rig, and one post-fix session completed the send (**estado: enviada**, artifact committed; not in the N=10 table).
- **Designed barge-ins never made the 500 ms window: 0/3** (1,338 / 9,034 / 20,501 ms) with interruption configured throughout — the same weakness that failed the 10 dB noise gate and drove the post-visit pivot.
- **Run-to-run variance persists** — one v4 session spoke read-backs without registering the timeline. v4 mitigates; it does not eliminate.

Judge-verifiable end to end

Reproducible

npm run smoke:mock — deterministic, no API key, no cost: services/timeline/action-items/severity precision & recall **100%** vs seeded ground truth, **4/4 seeded capture errors rescued** by the spoken loop, WER 0.006. Four real-session artifacts committed as evidence.

Private by code, not policy

Consent before the mic. Audio never uploaded or persisted — the artifact carries **audio_retained: false** and the server **rejects** anything claiming otherwise (25 backend self-checks).

What's next

Confirmation + new data in one turn (action items went 0/2 when the confirmation rule swallowed the new data) · variance reduction · verbatim-vs-normalized capture policy · mobile/headset validation.

Voice in, evidence out — the field-service record you can invoice against, because the values were read back out loud and the numbers (failures included) are on the table.

github.com/el-informatico/field-service-voice-logger · field-service-voice-logger.vercel.app · [docs/SUBMISSION.md](#) · [docs/evidence/gate/](#)